Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering & Technology
CSE-AIDS --- LEVEL 1
Capstone Project Synopsis 2026-27

Group No:	Students names	PRN	Email, Mobile number	Panel	Sign of student
	Saaransh Johri	1032233769	saaransh.johri@mitwpu.edu.in, 8586015349	B	
	Aditya Kiran	1032233722	aditya.kiran@mitwpu.edu.in, 9508557835	B	
	Vinit Dudhat	1032233356	vinit.dudhat@mitwpu.edu.in, 9021946877	B	
	Sayali Patil	1032233901	sayli.patil@mitwpu.edu.in, 7378355407	B	

Project Title: Autonomous Multi-Agent Merge Auditor & Execution-Sequence Verifier

Project Domain: Explainable AI (XAI), Trust & AI Safety

INHOUSE PROJECT: YES

Abstract:

Modern distributed software engineering relies heavily on version control systems like Git to merge parallel developer contributions into primary branch pipelines. However, standard Git merge tools rely almost entirely on line-by-line textual comparisons. Consequently, they fail to detect non-textual conflicts such as **semantic/logic incompatibilities** (e.g., modified function signatures called elsewhere), **out-of-order execution bugs** (e.g., initialized states moved across steps), and **side-effect collisions** (e.g., dual writes to global runtime states or database schemas).

To bridge this critical gap in CI/CD automation, MergeMind proposes an Autonomous Multi-Agent Merge Auditor and Execution-Sequence Verifier framework. MergeMind shifts merge conflict detection from raw text matching to a hybrid approach combining static program analysis, agentic reasoning, dynamic sandboxed execution, and Explainable AI (XAI). Upon pull request initialization across multiple incoming branches, the system triggers a coordinated workflow among specialized AI agents that analyze the codebase sequentially and collaboratively.
The process begins as an AST and Dependency Mapping Agent leverages language-agnostic parsers such as tree-sitter alongside Python's native ast and NetworkX to construct unified Abstract Syntax Trees and cross-file directed dependency graphs. Next, a Sequence and Logic Auditor Agent inspects these control-flow trees, checking parameter and import integrity while evaluating order-of-operations logic to identify out-of-sequence execution calls or signature mismatches across files.
The post-merge codebase is then passed to a Sandboxed Execution Agent, which spawns isolated execution environments via the Docker Engine SDK or E2B Sandbox to execute full unit and integration test suites dynamically.

If all agent audits pass successfully, MergeMind automatically approves and logs the merge into the primary pipeline. However, in the event of a failure, an XAI Diagnostic Agent processes AST diffs, runtime stack traces, and agent audit logs to generate human-interpretable diagnostic reports complete with visual dependency call traces, exact line-level failure breakdowns, and automated patch suggestions. By moving beyond primitive line-matching, MergeMind provides a robust safeguard against subtle post-merge regressions, with system performance systematically evaluated using key metrics including Semantic Conflict Recall Rate, False Positive Rate, and total Audit Execution Latency.

Hardware Requirements

- Processor: Intel Core i7 / AMD Ryzen 7 (10th Gen or higher) or Apple M-series Silicon.
- RAM: Minimum 16 GB (32 GB recommended for concurrent multi-agent and Docker runtime workloads).
- Storage: 512 GB SSD (Minimum 20 GB free space for Docker containers and dependency caching).
- GPU (Optional): NVIDIA GPU with CUDA support (if running local LLM backbones like Llama-3/Mistral).

Software & Framework Requirements

- Operating System: Linux (Ubuntu 22.04 LTS / 24.04 LTS) or macOS / Windows 11 (via WSL2).
- Programming Languages: Python 3.11+, JavaScript (for tree-sitter bindings).
- Agent Framework & Orchestration: LangGraph / PydanticAI / AutoGen, NetworkX.
- AST & Code Parsing: tree-sitter, Python ast module.
- LLM Engine: Claude 3.5 Sonnet / GPT-4o API (or local Ollama deployment).
- Execution & Containerization: Docker Engine SDK for Python / E2B Sandbox API.
- Frontend Dashboard: Streamlit or React.js (for dependency visualization and XAI trace reports).

High Level Design [Architecture diagram/block diagram]